

Mobile security checklist

Status: done. All five steps below have real, reproducible output in docs/scans/mobile/, generated 2026-09-22 once an Android SDK/emulator became available in this environment (apksigner/aapt/zipalign from <sdk>/build-tools/<version>; no apktool/jadx/mitmproxy were installed - see the notes under each step for what that does and doesn't affect). This file is kept as the reproducible step-by-step record; if you want to re-run any of it yourself, the commands below are exactly what was run.

No real classroom host was provided for this assignment. Per Rule 3 (synthetic test data only), the release build and its certificate pinning point at a locally-generated, throwaway self-signed CA instead of a real one - see backend/run_https.py and step 4 below.

0. Setup

1. Open mobile/Product_Search_Project in Android Studio (it will resolve the Gradle/AGP/Kotlin versions in gradle/libs.versions.toml on first sync), or just use ./gradlew from the command line.
2. backend/.env: GOOGLE_CLIENT_ID_MOBILE needs a **Web application** OAuth client id (Credential Manager's setServerClientId() requires one - see the README's Google Sign-In section; no Android-type client or SHA-1 registration is needed for this flow).
3. Generate the synthetic HTTPS host's certificate (see step 4) before building the release APK, since it's bundled into the APK as a raw resource (app/src/release/res/raw/lab_test_ca.pem).

1. Build and sign the release APK

```
cd mobile/Product_Search_Project
keytool -genkeypair -v -keystore release.jks -alias lab -keyalg RSA -keysize 2048 -validity 3650 \
-storepass labrelease123 -keypass labrelease123 \
-dname "CN=Lab Release, OU=SSE, O=IIT Hyderabad, L=Hyderabad, ST=Telangana, C=IN"
./gradlew assembleRelease
<sdk>/build-tools/<version>/apksigner sign --ks release.jks --ks-pass pass:labrelease123 \
--out app-release-signed.apk app/build/outputs/apk/release/app-release-unsigned.apk
<sdk>/build-tools/<version>/apksigner verify --print-certs app-release-signed.apk
```

Kept release.jks out of the submission ZIP (it's a secret, not source - already gitignored via mobile/**/*.*jks).

Two real bugs were found and fixed getting this to build at all (the release variant had never actually been compiled before this pass):

4. network_security_config.xml's release variant had a literal -- inside an XML comment, which is invalid XML and breaks compilation.
5. okhttp-logging-interceptor was debugImplementation-only, but ApiClient.kt (shared between both build types) references HttpLoggingInterceptor unconditionally - release compilation failed with "unresolved reference." Fixed by making the dependency available to both variants; R8 still correctly strips the if (BuildConfig.DEBUG) branch that uses it from the release build (confirmed in step 2 below).

2. Inspect the release APK for hardcoded secrets / debug flags

apktool/jadx (decompilers) weren't installed in this environment, so the APK was unzipped directly and its dex/resources searched with strings + grep instead - this reads the same underlying string table a decompiler would, just without the nicer formatting:

```
unzip app-release-signed.apk -d apk-inspect
cd apk-inspect
strings classes.dex resources.arsc | grep -iE \
"client_secret|GOCS PX|api[_]?key|BEGIN (RSA|EC) PRIVATE KEY|password.{0,3}=" \
| tee ../../docs/scans/mobile/apk_secret_scan.txt

<sdk>/build-tools/<version>/aapt dump badging app-release-signed.apk \
| grep -i "application-debuggable" # prints nothing -- see apk_debuggable_check.txt
<sdk>/build-tools/<version>/aapt dump xmltree app-release-signed.apk AndroidManifest.xml \
| grep -i allowBackup # allowBackup=0x0 (false)

# bonus: confirm R8 actually renamed the app's own classes
strings classes.dex | grep -c "ProductRepository\|AppViewModel\|GoogleAuthConfig" # 0
strings classes.dex | grep "MainActivity" # only manifest-declared activities survive renaming
```

Result: clean. No client_secret, GOCS PX- (Google's OAuth client-secret prefix), API keys, or PEM private keys anywhere in the APK - the only matches are Kotlin property *names* like password=, not values. Non-debuggable, allowBackup=false, and R8 minification confirmed active (0 matches for the app's own class names, vs. the one manifest-required MainActivity). Full write-up: docs/scans/mobile/apk_secret_scan.txt, apk_debuggable_check.txt.

3. Verify secure local storage

```
adb shell run-as com.example.product_search_project ls -la shared_prefs/ databases/
adb shell run-as com.example.product_search_project cat shared_prefs/secure_auth_prefs.xml
```

Result: secure_auth_prefs.xml (from TokenStore.kt's EncryptedSharedPreferences, AES256-SIV keys / AES256-GCM values) has both encrypted *keys* and encrypted *values* - no readable access_token/ refresh_token name, no plaintext JWT anywhere in the file, and no databases/ directory at all (no secondary plaintext cache). Combined with allowBackup=false above, both halves of the local-storage risk are closed: encrypted at rest, and the OS export path that could otherwise exfiltrate it is disabled. Full write-up: docs/scans/mobile/local_storage_check.txt.

4. MITM / TLS enforcement demonstration

No real classroom host, no mitmproxy install in this environment - so this uses a locally-generated self-signed CA as the "network presenting an unexpected certificate," which is the actual condition this control defends against (a real mitmproxy run is just one way to produce that same condition):

```
cd backend
mkdir -p certs
# primary (actively served) cert -- bundled into the release APK as its trust anchor
openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
-keyout certs/server.key -out certs/server.crt \
-subj "/CN=10.0.2.2/O=Product Search Lab/OU=Synthetic Test CA" \
-addext "subjectAltName=IP:10.0.2.2,IP:127.0.0.1"
cp certs/server.crt ../../mobile/Product_Search_Project/app/src/release/res/raw/lab_test_ca.pem

# backup cert -- pinned but never served, for certificate rotation
openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
-keyout certs/server-next.key -out certs/server-next.crt \
-subj "/CN=10.0.2.2/O=Product Search Lab/OU=Synthetic Test CA (backup)" \
```

```
-addext "subjectAltName=IP:10.0.2.2,IP:127.0.0.1"
```

```
# attacker cert -- NOT in the pin-set, stands in for a MITM proxy's substituted cert
openssl req -x509 -newkey rsa:2048 -nodes -days 3650 \
-keyout certs/attacker.key -out certs/attacker.crt \
-subj "/CN=10.0.2.2/O=Attacker MITM Proxy/OU=Not Pinned" \
-addext "subjectAltName=IP:10.0.2.2,IP:127.0.0.1"
```

```
# extract each cert's SHA-256 SPKI pin (goes in network_security_config.xml)
openssl x509 -in certs/server.crt -pubkey -noout | openssl pkey -pubin -outform der \
| openssl dgst -sha256 -binary | openssl enc -base64
```

```
python3 run_https.py # serves the app on :5443 with server.crt/key
```

Test sequence (full transcript in docs/scans/mobile/mitm_pinning_test.txt):

6. **Legitimate cert** (run_https.py running as above): release app logs in successfully over real TLS with pinning passing.
7. **Substituted cert** (swap run_https.py to serve attacker.crt/key on the same host:port instead): the exact same login attempt never reaches the server - javax.net.ssl.SSLHandshakeException: CertPathValidatorException: Trust anchor for certification path not found, observed against the real R8-obfuscated release build (stack frames like yk1.g/zk1.b, not readable class names). No credentials or data are sent before the handshake fails.
8. **Legitimate cert restored**: same login, same credentials, no reinstall - works again, confirming step 2's failure was specific to the substituted certificate, not a general break.

This demonstrates TLS enforcement (cleartextTrafficPermitted="false", no plaintext fallback), correct certificate handling (an untrusted/unpinned certificate is rejected at the trust-anchor level), and that no sensitive data leaks in transit under substitution (the handshake fails before any HTTP request is transmitted).

5. SBOM for the mobile app's dependencies

Real, tool-generated (not hand-authored) as of this pass, using the CycloneDX Gradle plugin (added to app/build.gradle.kts / gradle/libs.versions.toml):

```
cd mobile/Product_Search_Project
./gradlew cyclonedxBom
cp app/build/reports/cyclonedx/bom.json ../../docs/sbom/mobile-sbom.json
```

282 components - the full resolved dependency graph (direct + transitive), not just the ~19 direct entries in gradle/libs.versions.toml.